

Category 1 — Mutable Defaults (5 Traps)

From "Guess the Output — 50 Pythonic Traps" · Python Interview Prep 2026 · Learn & Excel

This category covers the single most reliable "is this candidate prepared?" filter in Python interviews. Default argument values are evaluated **once**, at function-definition time, and the same object is reused on every call. When that object is mutable — `list`, `dict`, `set`, or any custom mutable — every call mutates the same shared state. The bug looks like memory; the cause is binding-time. Five traps below: the classic list, the dict variant, the set variant, the workaround done wrong, and the class-attribute version that bites teams in production.

Trap 1 · The classic shared list

Snippet

```
def add_item(item, target=[]):
    target.append(item)
    return target

print(add_item('a'))
print(add_item('b'))
print(add_item('c'))
```

Your prediction: Write down what you think the three lines print *before* scrolling.

The actual output:

```
['a'] ['a', 'b'] ['a', 'b', 'c']
```

Why it does that:

The `target=[]` expression in the signature is evaluated exactly once, when Python compiles the `def` statement. The resulting empty list is stored on the function object itself — you can inspect it as `add_item.__defaults__[0]`. Every call that omits `target` does not get a fresh list; it gets a fresh *binding* to the **same** list that has lived on the function since module load.

When the first call appends `'a'`, it mutates that single list. The second call binds `target` to the same now-non-empty list, appends `'b'`, and so on. The list returned by every call is the literal same object — `add_item() is add_item()` evaluates to `True`. The function looks pure but is actually a singleton accumulator.

The canonical fix is the sentinel pattern: `def add_item(item, target=None): if target is None: target = []`. The `None` default is immutable, so re-binding `target` inside the body creates a fresh list per call. Use `is None`, not `if not target` — see Trap 4.

What saying this tells the interviewer: *"Default values are bound at def-time to the function object, so mutable defaults become shared state across calls — I always use `None` as the sentinel and allocate inside the body."*

Trap 2 · Dict default with `setdefault`

Snippet

```
def cache_user(user_id, name, cache={}):
    cache.setdefault(user_id, name)
    return cache

print(cache_user(1, 'Asha'))
print(cache_user(2, 'Rohan'))
print(cache_user(1, 'Vikram'))
```

Your prediction: Three dicts printed. What's in each, and does `Vikram` overwrite `Asha` ?

The actual output:

```
{1: 'Asha'} {1: 'Asha', 2: 'Rohan'} {1: 'Asha', 2: 'Rohan'}
```

Why it does that:

Same mechanism as Trap 1, but the failure mode is worse because it looks intentional. The `cache={} dict` is created once at def-time and is the *same dict* across all three calls. The first call inserts `1 → 'Asha'`. The second call sees the existing entry for `1` is missing for key `2`, so it adds `2 → 'Rohan'`. The third call hits `setdefault(1, 'Vikram')` — `setdefault` only writes when the key is absent, so `'Asha'` survives and `'Vikram'` is silently discarded.

This pattern shows up in real codebases as "lazy cache" attempts — a junior dev wanting per-call memoization but accidentally building module-level memoization with a default-argument dict. In a Django view or FastAPI handler, the dict accumulates across every request the worker process serves, which leaks memory and — worse — leaks user data between requests when keys collide.

What saying this tells the interviewer: *"This is the mutable-default trap with a dict — the cache is shared across calls, so in a web worker it doubles as a cross-request data leak."*

Trap 3 · Set default accumulating across calls

Snippet

```
def tag_event(event, seen=set()):
    seen.add(event)
    return seen

print(tag_event('login'))
print(tag_event('checkout'))
print(tag_event('login'))
```

Your prediction: Three sets. Does the third call see `'login'` as already-seen?

The actual output:

```
{'login'} {'login', 'checkout'} {'login', 'checkout'}
```

Why it does that:

The set literal `set()` in the default position constructs one set at def-time, attaches it to `tag_event.__defaults__`, and reuses it forever. `seen.add('login')` mutates that single object. The third call adds `'login'` again, which is a no-op because sets are unique — but the set itself is the same one from calls 1 and 2, now containing both `'login'` and `'checkout'`.

The dangerous twist with sets is that the bug is invisible when the function is "supposed to" track uniqueness anyway. A code reviewer skimming the function thinks "yes, sets dedupe, this looks correct." The trap only surfaces when a test runs the function twice in the same process — local dev passes, but the integration test where two scenarios share a worker fails with "why is checkout already in the seen-set for a fresh user."

For the fix, again use `seen=None` and allocate inside. If you genuinely want module-level accumulation, make it explicit: declare `SEEN_EVENTS: set[str] = set()` at module scope, so reviewers see "this is shared state" and treat it accordingly.

What saying this tells the interviewer: *"Same shared-default problem with `set()` — and it's harder to spot because uniqueness looks intentional."*

Trap 4 · The workaround that breaks on `[]`

Snippet

```
def append_safely(item, target=None):
    target = target or []
    target.append(item)
    return target

caller_list = ['existing']
print(append_safely('new', caller_list))
print(caller_list)

empty = []
print(append_safely('new', empty))
print(empty)
```

Your prediction: Four lines. Does the caller's list get mutated in both cases?

The actual output:

```
['existing', 'new'] ['existing', 'new'] ['new'] []
```

Why it does that:

This is the half-fix that senior interviewers love to ask about. The developer correctly used `None` as the default to dodge Trap 1, but then wrote `target = target or []` to handle the case where the caller passes nothing. The `or` short-circuits on falsy values, and **an empty list is falsy in Python**. So when the caller passes `[]`, the expression returns `[]` — but it's a *new* `[]`, not the caller's `[]`.

For the first call, `caller_list = ['existing']` is truthy, so `target = target or []` keeps the original reference. `append` mutates it in place; both prints show `['existing', 'new']`. For the second call, `empty = []` is falsy, so `target` gets rebound to a fresh new list. The mutation happens on the fresh list, the return shows `['new']`, but the caller's `empty` is untouched and still prints `[]`.

The fix is the explicit identity check: `if target is None: target = []`. This treats only the literal sentinel as "create new" and respects empty containers the caller deliberately passed. The same bug appears with empty strings, zero, and empty dicts — every codebase has at least one `value = arg or default` that silently misfires on falsy-but-valid inputs.

What saying this tells the interviewer: "`or` short-circuits on any falsy value, so `target or []` discards an empty list the caller deliberately handed in — the correct sentinel check is `is None`, never truthiness."

Trap 5 · Mutable default on `__init__` — every instance shares

Snippet

```
class Order:
    def __init__(self, items=[]):
        self.items = items

o1 = Order(); o1.items.append('book')
o2 = Order(); o2.items.append('pen')
o3 = Order()
print(o1.items, o2.items, o3.items)
print(o1.items is o2.items is o3.items)
```

Your prediction: What does each `Order` show as its items, and is the identity check `True` ?

The actual output:

```
['book', 'pen'] ['book', 'pen'] ['book', 'pen'] True
```

Why it does that:

`__init__` is a function like any other — Python doesn't give methods special default-argument treatment. The `items=[]` default is evaluated once at class-definition time, stored on the unbound `__init__` function, and shared across every instance that omits the argument. `self.items = items` then binds three different attribute names (`o1.items`, `o2.items`, `o3.items`) to the same list. Every `append` mutates that one list; the identity check confirms all three attributes point at it.

This is the production-incident version of the trap. The team ships a `Cart`, `Order`, `Form`, or `Config` class with a mutable default in `__init__`, tests pass locally because each test creates a fresh process, and then orders start showing up with line items from completely unrelated customers. In a long-running gunicorn or uWSGI worker, the shared list accumulates across every request the worker handles until it gets recycled.

The correct pattern is the sentinel inside `__init__`: `def __init__(self, items=None): self.items = items if items is not None else []`. For dataclasses, use `field(default_factory=list)` — the factory is called per-instance, not at class-definition. If you spot `items: list = []` on a dataclass, that's the same trap with a type annotation glued on; `@dataclass` has raised `ValueError` for mutable defaults on its fields since dataclasses were introduced in Python 3.7, but plain `__init__` still happily accepts them.

What saying this tells the interviewer: *"Methods aren't special — `__init__(self, items=[])` shares one list across every instance, so I reach for `default_factory=list` on dataclasses and the `None` sentinel everywhere else."*

Category 2 — Late-Binding Closures (5 Traps)

Python Interview Prep — 2026 Edition · Learn & Excel · File 05 Guess the Output

This is the second of the eleven trap categories that make up File 05. The five questions below all turn on a single Python rule: a closure captures the *name* of a variable, not the value the variable held when the closure was built. The name is resolved when the closure is called, not when it is created. Junior devs hit this once and assume it is a `for` loop bug. Round-1 panels at Razorpay, Cred, Swiggy, PhonePe, Goldman India, JP Morgan, Postman, and Walmart Labs lean on this rule because it separates candidates who have read the data model from candidates who have only written Python. Each trap below has the same shape: the snippet, your prediction, the actual output Python prints on 3.12+, the mechanism that produces it, and the one line you can say in the room that tells the interviewer you actually understand what is happening.

Trap 6 · The classic for-loop lambda

Snippet

```
funcs = [lambda: i for i in range(3)]  
print([f() for f in funcs])
```

Your prediction: Write it down before you scroll. `[0, 1, 2]` ? `[2, 2, 2]` ? Something else?

The actual output:

```
[2, 2, 2]
```

Why it does that:

Every lambda in the list captures the same name `i`. The name is not resolved when the lambda is built — it is resolved when the lambda is called. By the time the second list comprehension calls each `f()`, the original comprehension has finished, and `i` is sitting on its final value `2`. All three lambdas read the same cell and all three get back `2`.

The way to picture it: think of each lambda as carrying a sticky note that says "go look up `i` in the enclosing scope when you are called." It is not carrying a copy of `i`. Comprehensions in Python 3 do

have their own scope (unlike Python 2), so `i` is local to the comprehension that built the list. But that scope still has only one binding for `i`, and all three lambdas point to it.

The fix that idiomatic Python reaches for is default-argument capture — `[lambda i=i: i for i in range(3)]` — which evaluates the default at lambda-construction time and therefore freezes the value. That is Trap 8 below.

What saying this tells the interviewer: *"This is the late-binding rule — closures capture names, not values. I'd fix it with default-argument capture: `lambda i=i: i`. Same family of bug as the mutable default argument trap."*

That one sentence places you above the candidate who says "I think Python is doing something weird with the loop variable" and well above the candidate who runs it to find out.

Trap 7 · Same trap, with `def` instead of `lambda`

Snippet

```
funcs = []
for i in range(3):
    def f():
        return i
    funcs.append(f)

print([fn() for fn in funcs])
```

Your prediction: Does swapping `lambda` for `def` change anything? Many candidates assume `def` is "real" and therefore captures differently.

The actual output:

```
[2, 2, 2]
```

Why it does that:

`def` and `lambda` produce the same kind of function object — the only difference is that `def` gives the object a real `__name__` and lets you write multi-statement bodies. The closure machinery is identical. When the loop ends, `i` is `2`, and every `f` in the list reads the same cell.

There is a second wrinkle worth mentioning. A `for` loop in Python does not introduce a new scope. The loop variable `i` is a regular local of the enclosing function (or of the module, if you are at top level). The three `def` statements all run inside that single scope, so the three function objects all close over

the same `i` cell. If you wanted three separate bindings, you would have to wrap the body in something that does introduce a scope — like a factory function call:

```
def make_returner(value):
    def f():
        return value
    return f

funcs = [make_returner(i) for i in range(3)]
print([fn() for fn in funcs])    # [0, 1, 2]
```

Each call to `make_returner` gets its own `value` cell. The inner `f` closes over that fresh cell, and the trap is gone.

What saying this tells the interviewer: "Same trap. `def` and `lambda` close over identically. The fix is to introduce a fresh scope per iteration — either a factory function, `functools.partial`, or default-argument capture."

You will get a nod for naming three fixes. Most candidates name one.

Trap 8 · The default-argument fix

Snippet

```
funcs = [lambda i=i: i for i in range(3)]
print([f() for f in funcs])
```

Your prediction: This is the standard fix. Does it still feel like a hack, or do you know why it works?

The actual output:

```
[0, 1, 2]
```

Why it does that:

Default arguments are evaluated once, at the moment the `def` (or `lambda`) statement runs. That evaluation happens *eagerly* — before the function body is ever entered, and before the loop moves to the next iteration. So when the loop is on `i = 0`, the lambda `lambda i=i: i` is built with the default value of the parameter `i` set to the current `0`. The default is stored in the function object's `__defaults__` tuple — it is now a property of the function, not a lookup into the enclosing scope.

Compare the bytecode of `lambda: i` versus `lambda i=i: i` — both have a `LOAD_FAST` of `i` in the body, but only the second one has a default stored on the function object at construction time. The first one must look `i` up in the closure cell every time it is called.

Two subtleties worth knowing in the room. First, the parameter `i` of the lambda *shadows* the outer `i` — they are different bindings that happen to share a name. Inside the lambda, `i` is the parameter; outside it would be the closure variable (if the parameter were named anything else). Second, if a caller passes an argument — `funcs[0](999)` — the default is overridden and you get `999`, not `0`. The default protects the value only against the absence of an argument, not against deliberate override. In practice this rarely matters because nobody calls these lambdas with an argument.

What saying this tells the interviewer: "Default arguments are evaluated at `def` time, not at call time. That eager evaluation freezes the loop value into the function object's `__defaults__`, which is exactly what late-binding closures can't do."

That answer is the one that distinguishes "I memorised the trick" from "I understand the language semantics."

Trap 9 · Class-body comprehension reaching for a class-level name

Snippet

```
class C:
    base = 10
    items = [1, 2, 3]
    out = [base + x for x in items]

print(C.out)
```

Your prediction: Looks innocent — `base` and `items` are right there on the class, the comprehension uses both. `[11, 12, 13]` ?

The actual output:

```
NameError: name 'base' is not defined
```

Why it does that:

Class bodies are a separate scope from the comprehension that runs inside them. Python's scoping rules say nested functions and comprehensions do *not* see names defined in the enclosing class scope — they look straight past the class body and into the module or function that contains the class. The exception that catches everyone out: the *outermost iterable* of a comprehension is evaluated in the enclosing scope, not in the comprehension's own scope. That is why `for x in items` works —

`items` is looked up where the class body is, and finds the class-local `items`. But `base + x` is evaluated inside the comprehension's own scope, which cannot see `base`.

This rule has been in Python since 3.0 and trips people up because it feels arbitrary. The reasoning: comprehensions in Python 3 compile to an anonymous helper function under the hood (that is what gives them their own scope). Helper functions inside class bodies do not see class attributes — they would have to write `C.base` or `self.base`. So when the comprehension's body tries to read `base`, it does the same search a method body would do, and falls through to the module-level scope, where `base` is not defined.

The two clean workarounds: either bind `base` to a module-level name before the class definition, or sidestep the comprehension and use a `for` loop in the class body (which does not introduce its own scope), or write the comprehension as a `staticmethod` / `classmethod` and call it from `__init_subclass__` or `__init__`. The idiomatic answer in the room is: "I would not put the comprehension at class-body level — I would compute it in `__init__` or move `base` to a module constant."

It is fair to label this a scoping bug rather than a late-binding bug. The reason it lives in this section: the underlying mechanism is the same — Python resolves the name `base` lazily, at the point where the comprehension's body runs, not at the point where it is read by the parser. By the time the comprehension's body runs, the lookup is happening inside the comprehension's own scope, which never sees the class body.

What saying this tells the interviewer: "Class bodies don't propagate into nested scopes — comprehensions, lambdas, methods, all of them. The outermost iterable is the one exception, which is why `for x in items` works but `base + x` fails."

That one sentence pre-empts the follow-up "why does `items` work then" and signals you know the corner of the language well.

Trap 10 · Generator expressions and the outermost-iterable rule

Snippet

```
gens = [(x for x in [i]) for i in range(3)]
print([list(g) for g in gens])
```

Your prediction: Late-binding bites everything else, so you predict `[[2], [2], [2]]`. Right?

The actual output:

```
[[0], [1], [2]]
```

Why it does that:

This is where late-binding stops being uniform and starts looking like a contradiction. The same outermost-iterable rule from Trap 9 applies here, but it now *saves* you instead of biting you.

A generator expression compiles to a generator function under the hood. Like all comprehensions, the *outermost iterable* of that generator expression is evaluated immediately, at the point of construction, in the enclosing scope — not lazily on first iteration. So when the outer comprehension is on `i = 0`, the inner `(x for x in [i])` is built with `[i]` evaluated *right then*, producing `[0]`. That list is captured by value, not by reference, into the generator's local state. The same happens at `i = 1` and `i = 2`, giving three generators that each have their own frozen single-element list.

If you change the snippet so that `i` is used inside the body rather than as the iterable — say, `[(i for _ in range(2)) for i in range(3)]` — the late-binding trap comes back hard. That gives `[[2, 2], [2, 2], [2, 2]]`, because `i` is read lazily on iteration, and by the time you call `list(g)`, the outer loop has finished.

The takeaway is the asymmetry. Inside a comprehension or generator expression, the *first* `for ... in ...` evaluates its iterable eagerly in the enclosing scope. Every other `for` clause, every `if` clause, and the leading expression evaluate lazily in the comprehension's own scope. That asymmetry is the single rule that explains Trap 9's `NameError`, Trap 10's surprising rescue, and most of the "wait what" moments people have when their generator factory misbehaves.

Two debugging consequences. First, if a generator-of-generators gives you a value you did not expect, replace the inner `for` clause with a print statement and see whether you are looking at construction-time or iteration-time. Second, if a generator's data source is mutated between construction and consumption, the *outermost* iterable is safe (snapshotted) but any other reference is not.

What saying this tells the interviewer: *"The outermost iterable of a generator expression is evaluated at construction time, in the enclosing scope. Everything else — the body, the filter, the inner `for` clauses — is late-bound. That asymmetry is what makes this print `[[0], [1], [2]]` instead of `[[2], [2], [2]]`."*

If you can say that in one breath, the panel will move on. That is the goal.

End of Category 2 (5 traps · pages 6 – 17 of File 05). Category 3 — `is` vs `==` (small-int cache, interned strings, `is None` correctness) — picks up next.

Category 3 — `is` vs `==` (5 Traps)

Python Interview Prep — 2026 Edition · Learn & Excel · File 05 Guess the Output

The five traps in this category split confident Python writers from people who have read CPython source code. The line `if x == None` runs without an error and silences your linter if you turn the warning off — and in 99 cases out of 100 it does what you wanted. The hundredth case is the one the interviewer will set up: a custom `__eq__` that returns `True` against anything, an integer that just happens to be 257 instead of 256, a tuple literal that the compiler folded into a singleton until you pulled it through a constructor at runtime. Every trap below has been put on a whiteboard at a Razorpay / Swiggy / Flipkart / Goldman / Microsoft India panel in the last two years. The right answer in every case starts the same way: "`==` is value equality, `is` is identity — and identity comparisons against anything that is not a documented singleton (`None`, `True`, `False`, `NotImplemented`, the empty tuple) are an implementation detail of the interpreter, not a guarantee of the language."

A note on what the traps are testing. Most of the surprising results below come from one of three sources: (1) the CPython **small-int cache** of preallocated objects from `-5` to `256`, (2) the compiler's **literal folding** pass which collapses two identical literals in the same code block to one object, and (3) **string and tuple interning** which the interpreter is free to do for short, atom-like values. None of these are part of the language specification. PyPy, Jython, GraalPy, MicroPython and future CPython versions are all free to behave differently. The lesson the interviewer wants to hear is not the trick — it's the rule: **never use `is` to compare values; only use it to compare against singletons.**

Trap 11 · `256 is 256` vs `257 is 257` — The Small-Int Cache Boundary

Snippet

```
a = 256
b = 256
print(a is b)

a = int('257')
b = int('257')
print(a is b)
```

Your prediction: write down both values before reading on.

The actual output:

`True False`

Why it does that:

CPython preallocates one integer object for every value from `-5` to `256` inclusive at interpreter startup and reuses them for every reference to that value. The range was chosen because those values cover the vast majority of integer literals in real code — loop indices, status codes, list sizes, ages, small

counts — and the cache trades a few kilobytes of memory for skipping millions of `PyLong` allocations over the life of a program. When you write `a = 256; b = 256`, both names bind to the same cached object, so `a is b` is `True`. Memorise the boundaries exactly: `-5` is in, `-6` is out; `256` is in, `257` is out.

For the second pair, `int('257')` parses the string at runtime and allocates a fresh `PyLong` object for the result each time it is called. The two objects have the same value but different addresses, so `is` returns `False`. If you instead wrote `a = 257; b = 257` as bare literals in the same function body or module, the CPython compiler's literal-folding pass would collapse the two `257` literals into a single object — `a is b` would be `True` — but that is a compiler optimisation, not the cache, and it can disappear if the literals appear in different code blocks. Use `int('257')` in the demo to bypass folding and surface the real behaviour.

This is the trap that proves the rule. The same source code, with the only difference being a value of 256 versus 257, gives opposite identity answers. Any code that depends on the result is broken. Use `==` for value equality, always.

What saying this tells the interviewer: *"I know the range exactly, I know it is a CPython implementation detail, and I know the compiler's literal folding can hide the boundary in toy examples — which is why I never use `is` on integers in real code."*

Trap 12 · `"abc" is "abc"` vs A String Built at Runtime

Snippet

```
x = "abc"
y = "abc"
print(x is y)

p = "".join(["a", "b", "c"])
q = "".join(["a", "b", "c"])
print(p is q)
```

Your prediction: write down both values before reading on.

The actual output:

`True False`

Why it does that:

The first pair is `True` because CPython interns short, identifier-like string literals at compile time. When the bytecode compiler sees two `"abc"` literals in the same code block, it stores one string object in

the module's constants table and binds both names to it. The same is true for any string that looks like a Python identifier (letters, digits, underscore, length under roughly 4096 characters) — the compiler routes them through the interner. This is why naive `is` comparisons on string literals so often look correct: the compiler is silently making them work.

The second pair is `False` because `"".join([...])` builds a fresh string at runtime, character by character. The interpreter has no opportunity to consult the intern table during construction. Both calls produce strings with value `"abc"`, but they are two separate `PyUnicode` objects at two different addresses. `is` returns `False`. If you wanted both to point to the same interned object you would have to call `sys.intern(p)` and `sys.intern(q)` explicitly.

The general rule: **any string you constructed at runtime, read from a file, decoded from bytes, or assembled from user input is not interned.** Comparing two such strings with `is` is a coin flip that happens to land on `True` for short identifier-looking literals in toy code and `False` for everything else. Linters (`ruff`, `flake8`, `pylint`) all warn on `is` with a string literal for exactly this reason — `SyntaxWarning: "is" with 'str' literal. Did you mean "=="?` is the message CPython itself emits at compile time.

What saying this tells the interviewer: *"String interning is a compile-time optimisation that catches short identifier-like literals. The moment a string is built at runtime — `join`, `format`, `decode`, `input()` — interning stops, and `is` becomes unreliable. That is why `==` is the only correct way to compare string values."*

Trap 13 · `is None` vs `== None` — When `__eq__` Lies

Snippet

```
class AlwaysEqual:
    def __eq__(self, other):
        return True

x = AlwaysEqual()

print(x == None)
print(x is None)

if x == None:
    print("treated as None (DANGER)")
if x is None:
    print("really is None")
else:
    print("correctly rejected as not None")
```

Your prediction: write down all four lines before reading on.

The actual output:

```
True False treated as None (DANGER) correctly rejected as not None
```

Why it does that:

`==` invokes `__eq__`. The `AlwaysEqual` instance overrides `__eq__` to return `True` for every argument, including `None`. Python's comparison protocol calls `x.__eq__(None)` first, gets `True` back, and that's the answer — the engine never even gets a chance to ask `None` whether it agrees. The first print line is `True`, and the first `if` branch fires. From the calling code's point of view, `x` and `None` are indistinguishable. That is the bug.

`is` does not invoke `__eq__`. It is implemented as a pointer comparison in the CPython interpreter — literally `id(a) == id(b)` — and there is no way for any class to override or hook into it. `None` is a true singleton (one `NoneType` instance per interpreter, accessible as `type(None)()` returns the same object), so `x is None` is `True` if and only if `x` is the actual `None` object.

`AlwaysEqual()` is a different object, so `x is None` correctly returns `False` and the third `if` correctly rejects it.

The security implication is real and has shipped to production. A class that overrides `__eq__` to always return `True` — whether intentionally (a mock object in a test suite, a wildcard matcher, a sentinel "any value" class) or by mistake (a buggy `__eq__` written for a different equality model) — will pass every `== None` check in the codebase. If any of those checks gate authentication ("if user is None, reject"), authorisation ("if token is None, skip"), or sensitive operations ("if record is None, return 404"), the wrong instance can sail through. The same trap applies to `== True`, `== False`, `== 0` — all of them invoke `__eq__` and all of them can be lied to. Linters enforce `is None` / `is not None` over `== None` / `!= None` as a hard rule. PEP 8 codifies it. Code review should never let `== None` through.

What saying this tells the interviewer: "`== None` is overridable; `is None` is not. PEP 8 mandates `is None` and every Indian product co's security review will flag `== None` in code that handles auth, payments, or PII. The rule is not stylistic — it is a correctness rule that has caused real CVEs in libraries that trusted user-supplied objects."

Trap 14 · `[] is []` vs `() is ()` — The Empty Tuple Singleton**Snippet**

```
def make_empty_list():  
    return []  
  
def make_empty_tuple():  
    return ()  
  
print(make_empty_list() is make_empty_list())  
print(make_empty_tuple() is make_empty_tuple())  
print({} is {})  
print(frozenset() is frozenset())
```

Your prediction: write down all four values before reading on.

The actual output:

False True False False

Why it does that:

Every call to `[]` allocates a brand-new list. Lists are mutable, so the interpreter must give each caller a distinct object — otherwise mutating one caller's list would change everyone else's. Same for `{}` (mutable dict) and `frozenset()` (the constructor allocates a new frozenset each call even though the value is empty and could in principle be shared). All three return fresh objects on every call, and `is` returns `False`.

The empty tuple is the special case. CPython treats `()` as a documented singleton: there is exactly one empty tuple object per interpreter, and every `()` literal and `tuple()` constructor call returns that same object. The reason this is safe is that tuples are immutable — there is nothing a caller can do to a shared empty tuple to surprise another caller, because there are no mutating operations. The interpreter implements the optimisation in C: `PyTuple_New(0)` returns a cached interned object. So `make_empty_tuple() is make_empty_tuple()` is `True` even across function-call boundaries. This is one of the very few `is` comparisons on a non-`None` value that is documented and safe to rely on in CPython.

A nuance to mention: `()` is the only sized tuple that is a singleton. `(1,)` is *not* documented as a singleton — two calls to a function that returns `(1,)` may or may not return the same object depending on whether the compiler folded the literal into the constants table for that function. Bare-literal `(1,) is (1,)` in the same function body usually returns `True` because of literal folding, but `tuple([1]) is tuple([1])` returns `False` because the constructor allocates. Treat **only** the empty tuple as a singleton — every other tuple is fair game for `is` returning either answer.

What saying this tells the interviewer: *"Empty tuple is a CPython singleton because immutability makes sharing safe; empty list, empty dict, empty set are all freshly allocated on every call because mutating one shared empty container would corrupt every other caller. The rule generalises: the only safe `is`*

comparisons in CPython are against `None`, `True`, `False`, `NotImplemented`, and the empty tuple."

Trap 15 · `True == 1`, `True is 1`, and the Dict-Key Collision

Snippet

```
print(True == 1)
print(True is 1)

d1 = {True: "a", 1: "b"}
d2 = {1: "c", True: "d"}

print(d1)
print(d2)
print(len(d1), len(d2))
print(hash(True) == hash(1))
```

Your prediction: write down each of the six lines before reading on.

The actual output:

```
True False {True: 'b'} {1: 'd'} 1 1 True
```

Why it does that:

`bool` is a subclass of `int` in Python — `isinstance(True, int)` returns `True`, `True + 1` is `2`, and `True.bit_length()` works. The class was retrofitted in Python 2.3 to be a thin subclass of `int` rather than a separate type, which means `True` is an integer with value 1 for arithmetic and comparison purposes. So `True == 1` is `True` (value equality wins), and `True + True + True` is `3`.

`True is 1` is `False` because identity is not equality. `True` is a singleton instance of `bool`; `1` is a (cached) instance of `int`. They have different `id()` values — different objects at different addresses, with different runtime types. `is` reports the truth: they are two distinct objects that happen to compare equal.

The dict-key collision is the part interviewers love. Python dicts treat two keys as the same if `hash(k1) == hash(k2)` and `k1 == k2`. Because `bool` inherits `__hash__` from `int` and the hash of an integer is the integer itself (mod the dict's table size), `hash(True) == hash(1) == 1`. And `True == 1` is `True`. So as far as the dict is concerned, `True` and `1` are the same key. When you write `{True: "a", 1: "b"}`, the first entry inserts the pair with `True` as the key object and `"a"` as the value. The second entry finds a key-equal slot, **keeps the original key object** (`True`), and

overwrites the value with `"b"`. The result is `{True: 'b'}` — first-key-wins for the key, last-value-wins for the value. Reverse the insertion order and you get `{1: 'd'}` for the same reason. The dict has length 1 in both cases.

The same collision applies to `False` and `0` (also key-equal, same hash). It applies to `1` and `1.0` and `complex(1, 0)` (all key-equal because `1 == 1.0 == 1+0j` and they all hash the same). Any heterogeneous-key dict that mixes booleans with integers, or integers with floats, will silently merge them. Bugs found in the wild include: JSON-loaded configs where `True` from the file overwrote a numeric `1` key set in code; metrics dictionaries where `False` and `0` counted as the same bucket; cache keys that lost entries when an integer key was later inserted as a float.

What saying this tells the interviewer: "`bool` is a subclass of `int`, so `True` and `1` are value-equal and hash-equal — dicts collapse them into one entry, first-key-wins for the key, last-value-wins for the value. The fix is either to never use heterogeneous-type dict keys, or to use a strict equality check like `type(k) is bool` when you actually need to distinguish them. Same trap applies to `False / 0` and to `1 / 1.0 / 1+0j` because integer equality is value-based across numeric types."

Category 4 — Integer Caching (3 traps)

Python Interview Prep — 2026 Edition · Learn & Excel · File 05 Guess the Output

CPython pre-allocates a fixed pool of small integer objects at interpreter startup and reuses them every time you bind a name to one of those values. Any `int` literal in `range(-5, 257)` resolves to the same object in memory — so `is` (which compares identity, not value) returns `True` for those. Step outside that range and each occurrence is a brand-new object, so `is` returns `False`. None of this is a Python-language guarantee. It is a CPython implementation detail that PyPy, MicroPython, and Jython handle differently, and that even CPython has reserved the right to tune. The reason it shows up in interviews is that it lets the panel tell two things apart at once: do you understand that `is` and `==` are different operators, and do you know that `is` answers a question about the *interpreter's memory* rather than about *your data*?

The three traps below walk the boundary in three different ways. Trap 16 is the literal cache window. Trap 17 is the lower-bound asymmetry that everyone forgets. Trap 18 is the one that catches even people who know about the cache — because the compiler's constant-folder can hide the truth from you when you're typing two literals on adjacent lines.

Trap 16 · The 256 boundary — where the cache ends

Snippet

```
a = int("256")
b = int("256")
print(a is b)

a = int("257")
b = int("257")
print(a is b)
```

Your prediction: and

The actual output:

True False

Why it does that:

CPython's small-integer cache covers every integer from `-5` through `256` inclusive. When the interpreter starts up, it allocates exactly one `PyLongObject` for each of those 262 values and stashes pointers to them in a static array. Every time your code produces an integer in that range, the interpreter returns the cached object instead of creating a new one. Two names holding the same small integer therefore point at the same address, and `a is b` — which is just `id(a) == id(b)` at the C level — returns `True`.

`257` falls outside the cache. Each call to `int("257")` allocates a fresh `PyLongObject` on the heap, so the two `b` values live at different addresses and `a is b` evaluates to `False`. The values are equal (`a == b` is `True`), but the *objects* are not the same object.

A reasonable follow-up question: why use `int("256")` instead of just writing `a = 256; b = 256`? Because the compiler's constant-folder will rewrite literal `256` and literal `257` so aggressively that both pairs end up as the same object — even `257`. Constructing the integer at runtime via `int(...)`, arithmetic that doesn't fold, or pulling from an iterable bypasses the folder and lets the cache boundary actually show up. This is the rabbit hole Trap 18 lives in.

The whole pool sits in `Objects/longobject.c` in the CPython source, under the symbol `_PyLong_SMALL_INTS`. It has been that exact `-5..256` window since Python 2.5. The range was chosen because empirical analysis showed those are the integer values that dominate real Python programs — array indices, small counters, boolean-ish 0/1, return codes — so caching them is a real memory win.

What saying this tells the interviewer: *"That you can name the exact boundary (`-5` to `256`), that you know `is` compares addresses not values, and that you're aware constant-folding can make this trap look like it doesn't exist on `a = 257; b = 257` — which is why you reach for `int(...)` to prove the point. That's a senior-level answer to a question juniors think is about `is` vs `==`."*

Trap 17 · The lower bound — `-5` is in, `-6` is out**Snippet**

```
a = int("-5")
b = int("-5")
print(a is b)

a = int("-6")
b = int("-6")
print(a is b)
```

Your prediction: `True` and `True`**The actual output:**`True False`**Why it does that:**

The cache window is asymmetric — the lower bound is `-5`, not `-256` or `0`. Most candidates who know about the cache remember "up to 256" and assume the lower bound mirrors it. It does not. CPython caches `-5, -4, -3, -2, -1, 0, 1, 2, ..., 256`. That's 5 small negatives and 257 non-negatives, for the 262-value pool mentioned in Trap 16.

`-5` is in the pool, so both `int("-5")` calls return the same cached object — `is` returns `True`. `-6` is one step past the lower edge, so each `int("-6")` allocates a fresh object — `is` returns `False`. Identical mechanism to the upper boundary, just with a tighter and easier-to-forget threshold on the negative side.

Why `-5` specifically? It's the cheapest answer to "what negative integers do real Python programs use most?" The values `-1` (sentinel for "not found" in older Python C code), `0`, and small positives dominate. A handful of negative offsets show up in slicing (`list[-1]`, `list[-2]`). Beyond `-5` the frequency falls off a cliff, and the static-allocation memory cost stops paying for itself. So the CPython authors drew the line there and the rest of us have lived with it for two decades.

A real-interview anecdote: the asymmetry is the part the panel uses to separate "read a blog post about the small-int cache once" from "actually understands it." If a candidate says "Python caches small integers, that's why `256 is 256` is `True`," the standard follow-up is "what about negative integers?" The candidate who says "same way, all the way down to some boundary" is bluffing. The candidate who says `-5` is the lower bound is the one the panel writes down a check mark for.

What saying this tells the interviewer: *"That your knowledge of the cache is precise enough to name both bounds, that you noticed the asymmetry and didn't gloss over it, and that you can explain why the*

lower bound is tighter than the upper one (negative integers are rarer in real programs). This is the level of detail that puts you in the top quartile on an `is`-versus-`==` question."

Trap 18 · Same line, two literals — when the compiler hides the cache

Snippet

```
a = 257
b = 257
print(a is b)

a = int("257")
b = int("257")
print(a is b)
```

Your prediction: and

The actual output:

True False

Why it does that:

This is the trap that catches people who *know* the small-int cache. They look at the snippet and think "257 is outside the cache, so the first `print` is `False`." Wrong — on CPython 3.12+ both `print` statements give different answers, even though the integer value (`257`) is the same in both blocks.

The reason is the compiler's constant-folder, not the small-int cache. When CPython parses a code block, it builds a `co_consts` tuple of literal constants used in that block. If the same literal `257` appears twice on adjacent lines inside the same compilation unit (module body, function body, class body, or comprehension), the bytecode compiler folds them into a *single* `LOAD_CONST` reference into `co_consts`. Both names end up pointing at the *same* object — not because of the small-int cache, but because the compiler deduplicated the literal at parse time. So `a is b` is `True` even at `257`, `1000`, `10**9`, or `2**63 - 1`.

The moment you construct the integer at runtime — `int("257")`, `sum([256, 1])` via a non-foldable expression, integers pulled from a list, integers read from a file — the constant-folder is out of the picture. Each construction returns a fresh object, and `is` reflects the actual cache rule: `True` inside `[-5, 256]`, `False` outside. That's why the second pair returns `False`.

Two consequences worth saying out loud:

1. The classic trap snippet `a = 257; b = 257; a is b` returning `True` on a recent CPython is *not* evidence the cache extends past 256. It is evidence the compiler folded the literals. Different mechanism, accidental same result.
2. The behavior depends on the *compilation unit*. Type the two lines into a fresh interactive REPL session (each line is its own compilation unit, so no folding happens between them), and `a is b` will be `False` for `257`. Put them in a single function body or single script, and it's `True`. This is why old blog posts disagree with new blog posts — they were running the same code in different shells.

The interpreter version matters too. The exact constant-folding rules have shifted across CPython 3.7, 3.8, 3.10, 3.11, and 3.12 as the compiler gained new optimization passes. The cache *boundary* (-5 to 256) has held steady for two decades, but what the folder does on top of it has not. None of this is a Python-language guarantee — it's all CPython implementation detail. Run the same code on PyPy and you'll get a different answer, possibly on every line.

What saying this tells the interviewer: *"That you can name two separate mechanisms — the small-int cache (runtime) and the bytecode compiler's constant-folder (compile-time) — and explain why they produce overlapping but distinct behavior. That you know the REPL behaves differently from a script because of compilation units. And that you reach for `int(\"257\")` to disambiguate which mechanism is in play. This is the answer that ends the `is`-versus-`==` line of questioning and moves the interview forward."*

Category 5 — Float math (3 traps)

Python Interview Prep — 2026 Edition · Learn & Excel · File 05 Guess the Output

Three traps where the interviewer prints a float, an integer rounding result, or a running sum, and watches your face. None of this is Python being weird — it is IEEE-754 binary floating point doing exactly what the spec says it must, plus one rounding-mode choice the language made deliberately. **This is fundamental, not Python-specific** — Java, C, JavaScript, and SQL `DOUBLE PRECISION` all produce the same artefacts. The stand-out move on each of these is to name the underlying mechanism — binary representation, round-half-to-even, accumulated rounding error — before you reach for the fix.

Trap 19 · The classic — does `0.1 + 0.2` equal `0.3` ?

Snippet

```
a = 0.1 + 0.2
b = 0.3

print(a)
print(a == b)
```

Your prediction: Write down what you expect each `print` to produce. If you wrote `0.3` and `True`, this trap caught you. Read on.

The actual output:

```
0.30000000000000004 False
```

Why it does that:

The literals `0.1`, `0.2`, and `0.3` are *not* the decimal numbers you typed. Python parses them, hands them to the CPU's IEEE-754 double-precision encoder, and stores the closest binary fraction. `0.1` in binary is a repeating expansion — `0.0001100110011...` — that has to be truncated at 52 bits of mantissa. So the value actually stored for `0.1` is not 0.1 exactly; it is roughly `0.1000000000000000055511151231257827021181583404541015625`. `0.2` rounds up in the same way, and `0.3` rounds *down*. When you add the stored versions of `0.1` and `0.2`, the rounding errors compound, and you land one bit above the stored version of `0.3`. Hence the inequality.

Print the literals at 20 decimal places and the picture is obvious:

```
print('%.20f' % 0.1)      # 0.10000000000000000555
print('%.20f' % 0.2)      # 0.20000000000000000110
print('%.20f' % 0.3)      # 0.299999999999999998890
print('%.20f' % (0.1 + 0.2)) # 0.300000000000000004441
```

This is not a Python bug, a CPython quirk, or a 32-bit-vs-64-bit thing. It is the IEEE-754 double-precision standard — the same standard used by Java's `double`, C's `double`, JavaScript's `Number`, and your CPU's floating-point unit. Any language that stores reals as binary fractions has this artefact.

The fix: For comparison, use `math.isclose`:

```
import math
math.isclose(0.1 + 0.2, 0.3) # True
```

For exact decimal arithmetic (money, billing, tax), use `decimal.Decimal`:

```
from decimal import Decimal
Decimal('0.1') + Decimal('0.2') == Decimal('0.3')    # True
```

Note the strings inside `Decimal(...)`. `Decimal(0.1)` passes the *already-rounded* float in, so you get `Decimal('0.1000000000000000055511151231257827021181583404541015625')`.

Always construct decimals from strings, never from float literals.

What saying this tells the interviewer: "`0.1 + 0.2` is `0.30000000000000004` because IEEE-754 doubles cannot represent `0.1` exactly — they store the nearest binary fraction, and the rounding errors compound on addition. For comparison I would use `math.isclose`; for money I would use `Decimal` constructed from strings." — that one breath signals you know the representation layer, the comparison fix, and the money-arithmetic fix. Most candidates manage one of those three.

Trap 20 · `round(0.5)` is zero, but `round(1.5)` is two

Snippet

```
print(round(0.5))
print(round(1.5))
print(round(2.5))
print(round(3.5))
print(round(4.5))
```

Your prediction: Schoolhouse rounding says `1, 2, 3, 4, 5`. Write that down if that is your instinct. Then read on.

The actual output:

`0 2 2 4 4`

Why it does that:

Python's built-in `round` uses **banker's rounding** — formally, *round half to even*. When the value to be rounded is exactly halfway between two integers, the result is whichever of those two integers is even. So `0.5` rounds to `0` (even), `1.5` rounds to `2` (even), `2.5` rounds to `2` (even, not `3`), `3.5` rounds to `4`, `4.5` rounds to `4`. This is not a quirk Python invented — it is the default rounding mode mandated by IEEE-754 (`roundTiesToEven`), and it is what every serious financial and scientific library uses. The reason is bias: schoolhouse rounding (round half away from zero) systematically rounds positive halves *up*, which over millions of operations pushes accumulated sums upward. Banker's rounding averages out — roughly half the halves round up, half down, so the cumulative bias is zero.

The interview second-twist on this question is `round(2.675, 2)`. Most candidates expect `2.68` (banker's would say `2.68` since `2.68` is even in the second decimal). The actual result is `2.67`. The reason is not banker's rounding — it is Trap 19 again. `2.675` is not exactly representable in binary; the stored value is slightly *below* `2.675`, so it is no longer a true halfway case, and `round` correctly rounds it down. So `round` is doing two things you have to keep separate: it picks the nearest representable target by IEEE-754 rules first, *then* applies banker's rounding on actual ties.

The fix (if you specifically need schoolhouse rounding):

```
from decimal import Decimal, ROUND_HALF_UP
Decimal('0.5').quantize(Decimal('1'), rounding=ROUND_HALF_UP) # Decimal('1')
Decimal('2.5').quantize(Decimal('1'), rounding=ROUND_HALF_UP) # Decimal('3')
```

`Decimal` lets you pick the rounding mode explicitly — `ROUND_HALF_UP` (schoolhouse), `ROUND_HALF_EVEN` (banker's, default), `ROUND_DOWN`, `ROUND_CEILING`, etc. For money, banker's is normally what you want anyway; the GST/VAT systems in most jurisdictions specify it.

What saying this tells the interviewer: "`round` uses banker's rounding — round half to even — which is the IEEE-754 default and what every financial library uses, because schoolhouse rounding biases sums upward. And `round(2.675, 2)` returns `2.67` not `2.68`, because `2.675` cannot be stored exactly in binary, so it is not actually a halfway case at the representation level." — naming "banker's rounding" by its formal name, and connecting the `2.675` case back to binary representation, separates you from candidates who just memorised that "round behaves oddly in Python."

Trap 21 · Adding `0.1` ten times does not give you `1.0`

Snippet

```
total = 0.0
for _ in range(10):
    total += 0.1

print(total)
print(total == 1.0)
```

Your prediction: If you wrote `1.0` and `True`, this trap caught you. Write down what you actually expect before the reveal.

The actual output:

`0.9999999999999999 False`

Why it does that:

Same mechanism as Trap 19, scaled. Each `+= 0.1` adds the IEEE-754-rounded version of `0.1` (which is very slightly more than `0.1`) to the accumulator, but every intermediate sum gets re-rounded to fit in 52 mantissa bits. The rounding errors do not perfectly cancel — they accumulate. Across ten additions, the drift lands one bit *below* `1.0`. Run the same loop with 100 iterations and the drift is bigger. This is the classic accumulated-error pattern that ruins naive time-series sums, naive sensor averaging, naive bookkeeping ledgers, and anything else that adds floats in a loop.

A subtle wrinkle worth knowing if the interviewer pushes: `sum([0.1] * 10)` happens to round to `1.0` for these specific ten values — not guaranteed, just IEEE-754 luck on this exact input. Built-in `sum` does *not* use Kahan or Neumaier compensated summation; it walks the iterable doing plain `+=`, same as your loop. The reason the result lands at `1.0` here is that the per-addition rounding errors happen to cancel out on this particular sequence; scale to `sum([0.1] * 100)` and the drift is visible again. If the interviewer says "wait, my `sum` gave me `1.0`, you are wrong" — that is the answer. Point them at `sum([0.1] * 1000)` or the explicit loop, and the drift is visible.

The fix: For accurate float summation, use `math.fsum` — it implements the *Shewchuk* algorithm and returns the exactly-rounded sum:

```
import math
math.fsum([0.1] * 10)    # 1.0 — exactly
math.fsum([0.1] * 1000) # 100.0 — exactly
```

For exact decimal accumulation (money, ledgers), use `Decimal`:

```
from decimal import Decimal
sum(Decimal('0.1') for _ in range(10))    # Decimal('1.0')
```

For rational arithmetic that never loses precision at all, use `fractions.Fraction`:

```
from fractions import Fraction
sum(Fraction(1, 10) for _ in range(10))    # Fraction(1, 1) — i.e., exactly 1
```

`fsum` is the right pick for scientific summation; `Decimal` is the right pick for money; `Fraction` is the right pick for exact rational maths where you do not care about speed.

What saying this tells the interviewer: "Each `+= 0.1` adds a slightly-wrong float and then re-rounds — the errors accumulate, so after ten additions I land at `0.9999999999999999`. For accurate summation I would reach for `math.fsum`, which uses Shewchuk's compensated summation; for money I would use `Decimal`; for exact rational maths I would use `Fraction`." — naming `math.fsum` and Shewchuk's algorithm by name, and separating the three fix-tools by use case, is a senior-level answer. Most candidates know `Decimal` exists; the `fsum` reach is the differentiator.

Category 6 — List Aliasing (5 traps)

Python Interview Prep — 2026 Edition · Learn & Excel · File 05 Guess the Output

Aliasing is the single most-asked "your code looked right, but production was wrong" category. The bug is never in the line where you mutate — it's in the line where two names point at the same list object. This category covers the five shapes interviewers actually use: list-multiply, the explicit loop variant, chained assignment, mutable defaults on a class, and the shallow slice. Each one is a language guarantee, not a quirk. Each one ends careers when the grader reads `[[1], [1], [1]]` out loud and watches the candidate's face drop. Snippets compile and run on Python 3.12+.

Trap 22 · `[[[]] * 3]` and the list-multiply ambush

Snippet

```
grid = [[]] * 3
grid[0].append(1)
print(grid)
```

Your prediction: write down what you think `grid` looks like before you scroll.

The actual output:

```
[[1], [1], [1]]
```

Why it does that:

`[[[]] * 3]` does **not** build three independent empty lists. It builds **one** empty list object and then puts the same reference into the outer list three times. The multiplication operator on a list copies the *references* inside, not the objects they point to. So `grid[0]`, `grid[1]`, `grid[2]` are three names for the exact same inner list — the way `a` and `b` would be after `b = a`. When `.append(1)` mutates that shared object, every "row" sees the change because every row *is* the change.

Compare with `[[[] for _ in range(3)]]`. The comprehension evaluates `[]` on every iteration, which creates three distinct list objects, and the mutation stays local to whichever row you touched. The list-multiply trick is fine for immutable contents — `[0] * 3` is safe because integers can't be mutated in place — but the moment the inner element is itself a mutable container, the construction is a trap.

The interviewer will sometimes follow up with `[None] * 3` and ask why *that* one works. It works because you never mutate `None`; you only ever rebind `grid[i] = something`, which replaces the slot rather than touching the shared object.

What saying this tells the interviewer: "I read `list * n` as reference copies, not value copies — so I reach for a comprehension the moment the inner thing is mutable."

Trap 23 · The append-in-a-loop variant of the same bug

Snippet

```
row = [0] * 3
grid = []
for i in range(3):
    grid.append(row)
grid[0][1] = 9
print(grid)
```

Your prediction: write it down.

The actual output:

```
[[0, 9, 0], [0, 9, 0], [0, 9, 0]]
```

Why it does that:

Same bug, dressed up in a `for` loop so it looks innocent. `row` is created once, outside the loop, as a list object at some memory address. On each iteration, `grid.append(row)` appends the **same reference** to that one list. After three iterations, `grid` is a length-3 list whose three slots all point at the single `row` object. Reassigning `grid[0][1] = 9` walks the reference, finds the one shared list, and changes index 1 to 9. The other two rows are not other rows — they're the same row, viewed through two more names.

This shape catches candidates who memorized "don't use `[[[]] * 3`" but never internalized *why*. They'll happily write the loop version, convinced they fixed it because the multiplication is gone. The construction-time fix is to build each row inside the loop: `grid.append([0] * 3)`. Now `[0] * 3` evaluates fresh on every pass, giving you three independent row objects.

If you've already inherited the aliased structure from someone else's code and you need to break the aliasing without rewriting construction, the move is `grid = [r[:] for r in grid]` — but only if the inner elements are themselves not mutable containers. For nested mutables, you want `copy.deepcopy`. Which is the perfect bridge into Trap 26.

What saying this tells the interviewer: *"When I build a 2D structure, I make sure the inner object is constructed inside the loop or comprehension — never hoisted above it."*

Trap 24 · Chained assignment shares the object

Snippet

```
a = b = [1, 2, 3]
a.append(4)
print(b)
print(a is b)
```

Your prediction: write it down.

The actual output:

```
[1, 2, 3, 4] True
```

Why it does that:

`a = b = [1, 2, 3]` is not "create two lists." It's "evaluate the right-hand side once, then bind both `a` and `b` to that single value." So `a` and `b` are two names for one list object. `a.append(4)` is an **in-place** mutation — it walks the reference, finds the list, and tacks `4` onto the end. Reading `b` now finds that same mutated list, because `b` was never a separate object to begin with. `a is b` returns `True` because the `is` operator compares object identity, and there is exactly one object here.

The reason this surprises people is that the syntax looks symmetric with `a, b = 1, 2`, which *does* create independent bindings to independent integers. The difference is that chained assignment (`a = b = X`) targets the single value `X`, while tuple unpacking (`a, b = X, Y`) targets two separate values. Once you see the parse tree, the behavior is obvious — but Python lets you write the chained form so casually that the implication slips past.

Contrast this with rebinding: `a = [1, 2, 3]; b = a; a = a + [4]`. The third line evaluates `a + [4]`, builds a **new** list, and rebinds the name `a` to it. `b` still points at the original. So the rule of thumb interviewers want to hear: methods that mutate in place (`append`, `extend`, `+=` on lists, `sort`, `clear`) act on the shared object; operators that produce a new value (`+`, sorted, slicing) leave the original alone.

What saying this tells the interviewer: *"Chained assignment binds multiple names to one object — so in-place mutation through any of them is visible through all of them."*

Trap 25 · Mutable default argument shared across instances

Snippet

```
class Cart:
    def __init__(self, items=[]):
        self.items = items
    def add(self, x):
        self.items.append(x)

a = Cart()
a.add("apple")
b = Cart()
b.add("mango")
print(a.items)
print(b.items)
print(a.items is b.items)
```

Your prediction: write down what `a.items` and `b.items` contain.

The actual output:

```
['apple', 'mango'] ['apple', 'mango'] True
```

Why it does that:

Default arguments are evaluated **once**, at function-definition time, not each time the function is called. When Python reads `def __init__(self, items=[])`, it creates a single list object right there and stores it as the default. Every subsequent `Cart()` call that doesn't pass `items` falls back to **that same list**. So `a.items` and `b.items` are not two empty lists — they're two attribute names pointing at the one default list, which is now `["apple", "mango"]` because both instances appended to it.

This trap is brutal because it doesn't fail loudly. The first `Cart` you build looks fine. The second one inherits the first's data. By the time the third user logs in and sees the first two users' cart items, the bug is already in production. Test suites that build a fresh `Cart()` in each test case will pass individually and fail when run together, because the ordering of tests changes what the shared list contains.

The idiomatic fix is the sentinel pattern: default to `None`, then construct the real container inside the function body where it runs on every call.

```
class Cart:
    def __init__(self, items=None):
        self.items = [] if items is None else items
```

Now each `Cart()` gets a fresh list because `[]` is evaluated at call time, not at def time. The same rule applies to dicts, sets, and any custom mutable — never let a mutable be the literal default. If you want to be aggressive, set `mutable-default-argument` to error in Ruff or pylint; the linter catches it before code review does.

What saying this tells the interviewer: "Default arguments are evaluated at def time, so any mutable in that slot is shared across every call — I use the `None` sentinel and construct inside the body."

Trap 26 · Slice copies the outer list but not the inner ones

Snippet

```
a = [[1, 2], [3, 4]]
b = a[:]
b[0].append(99)
print(a)
print(a is b)
print(a[0] is b[0])
```

Your prediction: write down `a`, then both identity checks.

The actual output:

```
[[1, 2, 99], [3, 4]] False True
```

Why it does that:

`b = a[:]` does make a copy — but only of the outer list. The slice walks `a`, builds a new list of the same length, and fills it with the **same references** that `a` holds. So `b` is a distinct outer list (`a is b` is `False`), but `b[0]` and `a[0]` are still two names for the same inner list object (`a[0] is b[0]` is `True`). When you append `99` to `b[0]`, you're mutating that shared inner list, and `a[0]` — which points at the same object — shows the change.

This is the definition of a **shallow copy**. Every standard "copy" mechanism in the language behaves the same way at one level deep: `a[:]`, `a.copy()`, `list(a)`, `copy.copy(a)`, `[x for x in a]`. They all duplicate the container but not the contents. If the contents are immutable (ints, strings, tuples-of-ints), shallow is enough — you can't mutate them anyway. The moment a slot holds a mutable, shallow copy stops protecting you.

For nested mutables, you want `copy.deepcopy(a)`, which recursively builds new objects all the way down. It's slower, and it can't deep-copy file handles, sockets, or anything with a custom `__deepcopy__` that refuses. For the common 2D list case, `[row[:] for row in a]` is a fine

middle ground: outer list new, each inner row new, and you skip the recursion overhead. Just don't reach for it on 3-deep structures.

The interviewer's favorite follow-up is "what about `b = a.copy()` ?" The answer is "same shallow semantics — the list class's `copy()` method is documented as a shallow copy, equivalent to `a[:]` ." If they keep pushing, mention that `dict.copy()` , `set.copy()` , and `numpy.ndarray.copy()` are all shallow too; deep is opt-in across the standard library because the cost is real.

What saying this tells the interviewer: "Slice is a shallow copy — outer list is new, inner references are shared. For nested mutables I reach for `copy.deepcopy` or a per-row comprehension, depending on depth."

Category 7 — Comprehension Scoping (5 Traps)

Python Interview Prep — 2026 Edition · Learn & Excel · File 05 Guess the Output

Comprehensions look like syntactic sugar for a `for` -loop, and ninety percent of the time that mental model is fine. This category is about the other ten percent — the corners where a comprehension stops behaving like a loop because it secretly runs in its own function frame. That one detail (true since Python 3.0; people still forget) is the source of every trap below. The variable you wrote on the left is not the loop variable from the line above. The walrus you assigned inside the brackets escapes — but only because the language carved out an exception for it. A list comprehension dropped inside a class body cannot see the class's own attributes, while the same comprehension at module scope works fine. A nested comprehension reaches up to the enclosing comprehension's loop variable through closure, just like nested functions do — which is also why a list of generator expressions all snap to the same last value the moment you start consuming them.

Read each snippet as written. Predict the printed output. Then read the reveal. The point is not to memorise — the point is to be able to explain what the comprehension's own scope is doing the moment an interviewer puts a five-line snippet on the shared screen.

Trap 27 · The Loop Variable That Doesn't Leak

Snippet


```
i = 99
[i for i in range(3)]
print(i)
```

Your prediction: What does `print(i)` show after the comprehension runs?

The actual output:

99

Why it does that:

A list comprehension in Python 3 runs inside its own implicit function frame. The `i` written between the brackets is a local of that frame — it is allocated when the comprehension starts, bound on each iteration, and discarded the moment the comprehension returns. The `i = 99` on the outer line lives in the enclosing scope (module, function — whichever it is), and the comprehension never reaches up to rebind it. So the outer `i` is shielded; `print(i)` reads `99`.

This is the exact behaviour Python 3.0 introduced specifically to fix the Python 2 wart where the same snippet printed `2`. Generator expressions had their own scope even in Python 2; in 3.x, list / dict / set comprehensions joined them. If you want the last iterated value to survive, assign it explicitly (`last = i`) inside the comprehension using the walrus operator, or use a plain `for`-loop where leakage is the deliberate semantics.

What saying this tells the interviewer: "I know comprehensions run in their own function frame in 3.x — that's why the loop variable can't bleed into the enclosing scope the way it does in a regular `for`."

Trap 28 · The Walrus Escapes On Purpose

Snippet

```
result = [(y := x * 2, y) for x in [1, 2, 3]]
print(result)
print(y)
```

Your prediction: Does the second `print` raise `NameError`, or does it show a value?

The actual output:

```
[(2, 2), (4, 4), (6, 6)] 6
```

Why it does that:

The walrus operator (`:=` , PEP 572) was designed with one explicit exception to the rule from Trap 27: when used inside a comprehension or generator expression, it binds in the **enclosing** scope, not in the comprehension's own implicit frame. The reason is practical — the whole point of the walrus is to capture an intermediate value you want to reuse, and capturing it into a frame that gets torn down a millisecond later would defeat the purpose. So on every iteration `y` gets rewritten in the outer scope, the final write sticks, and after the comprehension finishes `y` is `6` (the last `x * 2`).

The rule has one sharp edge: if you try `[y := x for x in []]` , the comprehension still creates the outer binding *conceptually* but never assigns to it, so `y` will be undefined or whatever it was before. PEP 572 also bans the walrus in a handful of specific positions — most famously, you can't use it as the default value of a function parameter, so `def f(x = (n := 10)): ...` is a `SyntaxError` . The carve-out for comprehensions is targeted; outside those, the walrus has the same restrictions as any other assignment expression.

What saying this tells the interviewer: *"Walrus inside a comprehension is the one assignment that intentionally leaks — PEP 572 wrote that carve-out so the captured value is usable afterwards."*

Trap 29 · Class Body Plus List Comp Equals NameError

Snippet

```
class C:
    x = 5
    ys = [x for _ in range(3)]

print(C.ys)
```

Your prediction: Does this print `[5, 5, 5]` , or fail with `NameError` ?

The actual output:

```
NameError: name 'x' is not defined
```

Why it does that:

This is the most surprising consequence of "comprehensions have their own function scope." A class body is itself a namespace, but it is **not** an enclosing scope that nested functions can look up into.

Method bodies inside a class can't see other class attributes by bare name either — they reach them through `self` or `cls`. The same rule applies to the comprehension's hidden function frame: it looks for `x` in its own locals (not there), then in the enclosing function scopes (none, because the class body isn't one), then in module globals (not there), and finally raises `NameError`.

The fix is to drag the name in through an iterable that *is* evaluated in the class body. The outermost iterable of a comprehension is evaluated in the surrounding scope — that's the one place the class body's name lookup still works. So `ys = [x for _ in [x] * 3]` or `ys = [val for val in [x] * 3]` both succeed because `[x] * 3` is built before the comprehension's inner scope kicks in. Cleaner still, hoist the value to a local first or just write a regular `for`-loop in the class body.

What saying this tells the interviewer: *"A comprehension's implicit function frame can't see class attributes, so anything other than the outermost iterable has to reach `x` through a different mechanism."*

Trap 30 · Nested Comprehensions Reach Through Closure

Snippet

```
result = [[y for y in range(x)] for x in range(3)]
print(result)
```

Your prediction: Will the inner `range(x)` see the outer `x`, or fail with `NameError` the way Trap 29 did?

The actual output:

```
[[], [0], [0, 1]]
```

Why it does that:

Each comprehension has its own function frame, yes — but those frames nest the same way regular nested functions do, and the inner frame can read names from the outer frame through ordinary lexical closure. The outer comprehension binds `x` in its own frame; the inner comprehension is created *inside* that frame, so when it looks up `x`, the lookup walks one scope up, finds it, and uses it. That's why each row of the result corresponds to `range(0)`, `range(1)`, `range(2)`.

Contrast this directly with Trap 29: comprehensions chain to each other through enclosing-function-scope rules, and chaining works. A class body, by contrast, is not a function scope and is invisible to nested function lookups, so the chain breaks the moment one of the links is a class body instead of a function. The same `[expr for ... for ...]` reads the outer loop variable for the same reason — multiple `for` clauses live in the same single comprehension frame.

What saying this tells the interviewer: *"Nested comprehensions chain like nested functions — closure handles the lookup, which is exactly why the class-body version fails and this one works."*

Trap 31 · Generator Expressions Capture Late

Snippet

```
gens = [(x for _ in range(2)) for x in range(3)]
print([list(g) for g in gens])
```

Your prediction: *What does each generator yield when you finally consume it?*

The actual output:

```
[[2, 2], [2, 2], [2, 2]]
```

Why it does that:

A generator expression is a generator object — it doesn't run when you build it, it runs when you call `next()` on it. The `x` inside the generator is a free variable: it isn't bound in the generator's own frame, so at iteration time the generator looks it up by closure in the enclosing comprehension's frame. By the time the outer list comprehension finishes, that frame's `x` is `2` (the last value from `range(3)`). All three generators close over the **same** `x` and they read it at consumption time, so all three yield `2` twice.

You can see the same late-binding bug with `lambda` in a loop. The fix follows the same shape: bind the value eagerly at generator-construction time. The most idiomatic option is a default argument inside an inner factory — `[((lambda x=x: (x for _ in range(2))))() for x in range(3)]` — which snapshots `x` at the moment the lambda is called. Cleaner is to realise the inner sequence eagerly (`[x for _ in range(2)] for x in range(3)`) so there's no generator left to be lazy with. Both produce `[[0, 0], [1, 1], [2, 2]]`.

What saying this tells the interviewer: *"Generator expressions resolve free variables lazily through closure, so a list of them built in a loop will all snap to the loop's final value the moment you start consuming them — same shape of bug as `lambda` in a loop, same shape of fix."*

Category 8 — Iterator Exhaustion (5 Traps)

Python Interview Prep — 2026 Edition · Learn & Excel · File 05 Guess the Output

Every trap in this category turns on one fact that Python's syntax goes out of its way to hide: most of the things you iterate over only run *once*. A list can be looped a thousand times. A generator, a `zip`, an `enumerate`, a `map`, a `filter`, an opened file — these are one-shot. You walk to the end, the iterator's internal pointer parks at `StopIteration`, and every subsequent `for` or `list(...)` over the same object sees nothing. The bug is rarely the iterator itself — the bug is that the candidate *thought* they were holding a sequence and they were actually holding a cursor. Interviewers reach for this category because the failure mode is invisible to the type system (an exhausted generator and a fresh one have the same `repr`, the same `type`, and the same `__iter__` method), so the only defence is knowing which Python builtins return iterators vs sequences — by reflex, not by reasoning. The 5 traps below cover the five places this hits hardest in real code: a generator looped twice, `zip` short-circuiting on the shortest leg, a single iterator passed to `zip` as both arguments, `enumerate` re-listed, and a file handle iterated after its first pass.

Each trap below uses the standard Guess-the-Output shape. Read the snippet, lock in your prediction *before* you peek, then read the reveal and the mechanism. The "What saying this tells the interviewer" line is what you actually say out loud when this lands in front of you on a screen-share, so the interviewer hears the right vocabulary and moves on to the next question instead of probing whether you got it right by accident.

Trap 32 · Iterating a Generator Twice

Snippet

```
g = (x for x in range(3))
print(list(g))
print(list(g))
```

Your prediction: write down what you think the two `print` calls output before you scroll.

The actual output:

```
[0, 1, 2] []
```

Why it does that:

A generator expression returns a **generator object**, not a sequence. The generator object is itself an iterator — it has a `__next__` that yields values one at a time and a hidden cursor that advances every time `__next__` runs. The first `list(g)` pulls values until `StopIteration`, which leaves the cursor permanently parked at the end. The second `list(g)` calls `iter(g)` and gets the same already-exhausted generator back (a generator's `__iter__` returns `self`), so `__next__` raises `StopIteration` on the first call and `list` returns the empty list.

There is no "rewind". A generator is single-pass by design — that is what makes it memory-efficient over `range(10_000_000)`. If you need to iterate twice, either materialise once with `data = list(g)` and loop `data`, or rebuild the generator from its source with `g = (x for x in range(3))` each time you need a fresh pass. A common Indian-MNC interview riff is "why doesn't Python warn me when I iterate an exhausted generator?" — because an empty iteration is a valid program; the language can't tell the difference between "I meant to skip this" and "I expected three rows".

What saying this tells the interviewer: "Generators are one-shot iterators. After the first `list(g)` exhausts it, the second one prints `[]` — same object, cursor at EOF. If I need a second pass I either re-create the generator or materialise to a list."

Trap 33 · `zip` Short-Circuits on the Shortest Iterable**Snippet**

```
pairs = list(zip([1, 2, 3], [4, 5]))
print(pairs)
```

Your prediction: write down the list before you scroll. Look hard at the `3`.

The actual output:

```
[(1, 4), (2, 5)]
```

The `3` is gone. No error, no warning, no `None`, no padding — just silently dropped.

Why it does that:

`zip` pulls one item from each argument per iteration and yields a tuple. The instant any argument raises `StopIteration`, `zip` stops. With `[4, 5]` shorter than `[1, 2, 3]`, the second iterator runs out first; `zip` halts and never touches `3`. This is the single most common cause of "my dashboard is missing the last row" bugs in Python data pipelines — two parallel lists drift in length by one, and the join silently truncates to the shorter side.

The fix depends on intent. If equal length is a contract, use `zip(a, b, strict=True)` (Python 3.10+, PEP 618) — mismatched lengths now raise `ValueError`, which surfaces the bug at the join site instead of three layers downstream. If you genuinely want to pad the shorter side, use `itertools.zip_longest(a, b, fillvalue=None)`, which keeps going until the longest argument is exhausted and fills missing slots with the sentinel of your choice.

What saying this tells the interviewer: " `zip` stops at the shortest iterable — the `3` is silently dropped. In production code I'd use `zip(..., strict=True)` so a length mismatch raises instead of truncating, or `itertools.zip_longest` if I actually want the longer side preserved."

Trap 34 · `zip(it, it)` — The Same Iterator Twice

Snippet

```
it = iter([1, 2, 3])
print(list(zip(it, it)))
```

Your prediction: most candidates write `[(1, 1), (2, 2), (3, 3)]`. Lock yours in.

The actual output:

```
[(1, 2)]
```

One tuple. `3` is gone, and the pairing is `(1, 2)`, not `(1, 1)`.

Why it does that:

`zip` does not see "a list `[1, 2, 3]` reused twice". It sees two argument slots, both holding a reference to the *same iterator object*. Per iteration, `zip` calls `next()` on the first argument, then `next()` on the second argument, and packs the results into a tuple. Because both arguments point at the same `it`, the first `next` pulls `1`, the second `next` pulls `2`, and `zip` yields `(1, 2)`. On the next iteration, `next` on the first argument pulls `3`, `next` on the second argument hits `StopIteration`, `zip` halts. Total output: `[(1, 2)]`.

This is the textbook "chunk a flat list into pairs" idiom — `list(zip(*[iter(lst)] * 2))` rounds an even-length list into 2-tuples, `[(0,1), (2,3), (4,5), ...]`. It works precisely *because* the same iterator is shared across all argument slots. The same idiom written naively — `list(zip(iter(lst), iter(lst)))` — would create two independent iterators and give the wrong, doubled output. The trap is reading `zip(it, it)` as "zip the list with itself" instead of "advance the same cursor twice per tuple".

What saying this tells the interviewer: "Same iterator object passed twice means `zip` advances the one cursor twice per tuple — so I get `(1, 2)`, not `(1, 1)`. This is exactly how the `zip(*[iter(lst)] * n)` chunking idiom works."

Trap 35 · `enumerate` Is a One-Shot Iterator

Snippet

```
e = enumerate([1, 2, 3])
print(list(e))
print(list(e))
```

Your prediction: write both outputs before scrolling. The first one is the easy half.

The actual output:

```
[(0, 1), (1, 2), (2, 3)] []
```

Why it does that:

`enumerate` is not a wrapper that adds indices to a sequence. It is a builtin **iterator** type — `type(e)` is `<class 'enumerate'>`, and like every iterator in the standard library, it has a single internal cursor. The first `list(e)` walks the cursor to the end of the underlying `[1, 2, 3]`. The second `list(e)` finds the cursor parked at `StopIteration` and produces `[]`. Same failure mode as the generator in Trap 32 — different surface, identical mechanism.

`map`, `filter`, `reversed`, `zip`, `iter`, `open` (when treated as line iterator) and every generator-returning function in `itertools` share this shape. They are *iterators*, not *iterables-that-can-be-walked-many-times*. The list, tuple, dict, set, range, and string types are the opposite — they are iterables that hand out a *fresh* iterator every time you `for` over them, which is why you can loop a list 100 times and see the same items each time. The distinction is invisible to type hints (both satisfy `Iterable[T]`), so the only way to know which side a given builtin sits on is to learn the list.

What saying this tells the interviewer: "`enumerate` returns an iterator, not a sequence. First `list` exhausts it, second `list` returns empty. Same for `map`, `filter`, `zip`, `reversed` — they all hand out a one-shot cursor. If I need a re-iterable version I wrap with `list(enumerate(...))` once and re-loop that."

Trap 36 · File Handle Iterated After EOF

Snippet

Given a file `lines.txt` with three lines `alpha`, `beta`, `gamma`:

```
with open("lines.txt") as f:
    first = [line.rstrip() for line in f]
    second = [line.rstrip() for line in f]
print("first :", first)
print("second:", second)
```

Your prediction: write down `first` and `second` before you scroll.

The actual output:

```
first : ['alpha', 'beta', 'gamma'] second: []
```

Why it does that:

An open text file in Python implements the iterator protocol on top of the underlying OS file descriptor's read position. The first comprehension iterates the file object, which calls `__next__` repeatedly until the kernel returns EOF; the file pointer is now parked at the end of the file. The second comprehension iterates the same file object — `__iter__` returns `self`, `__next__` immediately observes EOF, and the comprehension finishes with an empty list. No exception, no warning, just a silently empty second pass.

This is a real production bug, not a trick question. It shows up the moment someone refactors a "read once" script into a "read header, then read body" function, or adds a debug `for line in f: print(line)` above the real loop, or writes a unit test that asserts on the body after the test fixture already consumed the file. The fix is one of three: (1) seek back with `f.seek(0)` if you genuinely need a second pass on the same handle, (2) materialise once with `lines = f.readlines()` and re-loop the list, or (3) open the file twice (cheap on local FS, expensive on S3 / network FS — pick deliberately). On `sys.stdin` and pipe-backed file objects, `seek(0)` will raise `io.UnsupportedOperation`; for those the only options are materialise-once or restructure the read so a single pass does the whole job.

What saying this tells the interviewer: "A file object is an iterator whose cursor is the OS read position. First loop walks to EOF, second loop sees EOF immediately and yields nothing. I'd either `f.seek(0)` to rewind, `readlines()` to materialise, or restructure to a single pass — and on `stdin` or a pipe, `seek` isn't available so single-pass is the only correct shape."

Category 9 — Class vs Instance Attributes (5 Traps)

Python Interview Prep — 2026 Edition · Learn & Excel · File 05 Guess the Output

Class-vs-instance attribute confusion is the bug that most "I know OOP" candidates eventually hit. The data model has two clean rules — reads walk `instance.__dict__` then `type(instance).__mro__`; writes always land on the instance unless you go through the class explicitly — and almost every trap in this category is a consequence of one of those two rules behaving exactly as documented while the candidate's mental model expected something else. The 5 traps below cover the canonical surfaces: mutable class attribute shared across instances, instance-level shadow created by reassignment, MRO read-through when a parent's class attribute changes, `__dict__` lookup precedence, and the `+=` -rebinds-vs- `.append()` -mutates fork that catches even people who already know about mutable defaults. Each trap follows the same shape: the snippet, your prediction prompt, the actual output, the mechanism, and the line you say in the interview that signals you know how Python's attribute lookup actually works under the hood.

Trap 37 · Mutable class attribute, shared list across instances

Snippet

```
class Cart:
    items = []

    def __init__(self, owner):
        self.owner = owner

a = Cart("ananya")
b = Cart("bharat")
a.items.append("notebook")

print(a.items)
print(b.items)
print(Cart.items)
print(a.items is b.items)
```

Your prediction: write down all four lines before reading on. The trap is whether `b.items` is empty or not, and what `Cart.items` shows.

The actual output:

```
['notebook'] ['notebook'] ['notebook'] True
```

Why it does that:

`items = []` at the class body level creates *one* list object owned by the class — not a fresh list per instance. Neither `a` nor `b` has an `items` key in its own `__dict__`. When you write `a.items`, Python finds nothing on the instance, walks up to the class, and returns the class-level list.

`a.items.append("notebook")` mutates that single shared list object in place. `b.items` then resolves the same way — instance miss, class hit — and returns the same mutated list. `Cart.items` is of course the same object. The `is` check confirms it: `a.items` and `b.items` are not two equal lists, they are literally the same list at the same memory address.

The fix is to move the assignment into `__init__`: `self.items = []` runs once per instance and gives each `Cart` its own list. This is the same root cause as the mutable-default-argument trap — the mutable object is created once at class-definition time, not per call or per instance.

What saying this tells the interviewer: *"A mutable class attribute is a shared singleton — every instance points at the same object until someone explicitly rebinds. The fix is `self.items = []` in `__init__`, same family of bug as mutable default arguments."*

Trap 38 · Instance assignment shadows the class attribute

Snippet

```
class Config:
    timeout = 30

a = Config()
b = Config()
a.timeout = 5

print(a.timeout)
print(b.timeout)
print(Config.timeout)

c = Config()
print(c.timeout)
```

Your prediction: did `a.timeout = 5` change the class attribute, or only `a`? What does a fresh instance `c` see?

The actual output:

5 30 30 30

Why it does that:

Writes through an instance never travel up to the class. `a.timeout = 5` inserts the key `"timeout"` into `a.__dict__` with value `5` and leaves `Config.timeout` untouched. From this point on, reading `a.timeout` finds the key on the instance and returns 5 without ever consulting the class. `b.timeout`, `Config.timeout`, and any future instance like `c.timeout` still resolve through the class fallback and return the original 30, because the class object itself was never modified.

This asymmetry — reads walk up, writes stay put — is the single most important rule in Python's attribute model. If you wanted to actually change the default for everyone, you had to write `Config.timeout = 5`. The bug that comes from this is usually subtler than the snippet: someone writes `self.config.timeout = 5` inside a method, expecting it to be a per-instance override, then is surprised when a teammate writes `Config.timeout` somewhere else and gets a different number than the running instance shows.

What saying this tells the interviewer: *"Instance assignment always creates a shadow in `instance.__dict__` — it never mutates the class. Reads fall through to the class only when the instance dict misses; once you've assigned, the class attribute is dead to that instance."*

Trap 39 · MRO read-through: changing a parent attribute updates unshadowed children

Snippet

```
class Base:
    region = "IN"

class Mumbai(Base):
    pass

class Delhi(Base):
    region = "IN-DL"

class Bangalore(Base):
    pass

Base.region = "IN-MH"

print(Mumbai.region)
print(Delhi.region)
print(Bangalore.region)
```

Your prediction: which of `Mumbai.region`, `Delhi.region`, `Bangalore.region` change when `Base.region` is reassigned?

The actual output:

`IN-MH IN-DL IN-MH`

Why it does that:

Class attribute lookup is itself an MRO walk. When Python evaluates `Mumbai.region`, it checks `Mumbai.__dict__` (no `region` key), then walks `Mumbai.__mro__` which is `(Mumbai, Base, object)`, finds `region` on `Base`, and returns the current value. `Mumbai` does not own a `region` attribute at any point — it borrows `Base`'s by reference, so anything that changes `Base.region` is immediately visible through `Mumbai`. Same for `Bangalore`.

`Delhi` is different because the class body wrote `region = "IN-DL"`, which inserted `"region"` directly into `Delhi.__dict__`. That entry shadows the inherited one, and the MRO walk for `Delhi.region` stops at `Delhi` itself — it never consults `Base`. Reassigning `Base.region` has no effect on `Delhi` because Delhi was never reading from Base in the first place.

The mechanism is consistent with Trap 38: shadowed names take precedence over inherited names, whether the shadow is on an instance shadowing a class or a subclass shadowing a parent class. The lookup path is the same shape — walk the dictionaries in MRO order, return on first hit.

What saying this tells the interviewer: *"Subclasses inherit class attributes by lookup, not by copy. Reassigning the parent's attribute updates every subclass that didn't redefine it. If you want a frozen snapshot per subclass, you have to assign it in the subclass body explicitly."*

Trap 40 · `__dict__` shadowing — instance dict always wins on lookup

Snippet

```
class Account:
    balance = 0

a = Account()
print(a.balance)

a.__dict__["balance"] = 1000
print(a.balance)
print(Account.balance)

del a.__dict__["balance"]
print(a.balance)
```

Your prediction: does writing directly into `a.__dict__` produce the same effect as `a.balance = 1000` ? What does the read return after the `del` ?

The actual output:

```
0 1000 0 0
```

Why it does that:

`a.balance = 1000` and `a.__dict__["balance"] = 1000` are functionally the same operation for an ordinary class — the assignment statement is just sugar that calls `__setattr__`, whose default implementation writes the key into the instance dict. So inserting into `__dict__` directly behaves identically: subsequent `a.balance` reads hit the instance dict first, find the key `"balance"`, return 1000, and never consult the class. `Account.balance` of course still returns 0 because the class object was untouched (same asymmetry as Trap 38).

The `del` half is the part that confuses people. Removing the key from `a.__dict__` does not delete the attribute as far as `a.balance` reading is concerned — it only removes the *shadow*. Once the shadow is gone, the lookup falls back through the MRO and finds `Account.balance = 0` again. So the attribute appears to "reset" to the class default. This is the legitimate way to clear an instance-level override and restore the class fallback, and it's how some descriptor protocols (notably `@cached_property`) work — they populate `instance.__dict__` so that future reads skip the descriptor entirely, and clearing the cache means just `del` ing the key.

(Caveat: if the class defines a data descriptor like `@property` or `__slots__`, the lookup order changes — data descriptors beat the instance dict. The default class here has neither, so the instance dict wins.)

What saying this tells the interviewer: "`obj.attr = value` is sugar for `__setattr__` which writes into `obj.__dict__`. Reading walks instance dict first, then the MRO, so `del obj.__dict__[\"attr\"]` is how you uncover the class-level fallback. This is exactly the mechanism `@cached_property` exploits."

Trap 41 · `append` mutates the shared list; rebinding creates an instance shadow

Snippet

```

class Bucket:
    items = []

a = Bucket()
b = Bucket()
a.items.append("apple")
print(a.items, b.items)
print("items" in a.__dict__)

Bucket.items = []

c = Bucket()
d = Bucket()
c.items = c.items + ["mango"]
print(c.items, d.items, Bucket.items)
print("items" in c.__dict__)

```

Your prediction: in the first block, does `b.items` show `["apple"]` or stay empty? In the second block, does `d.items` show `["mango"]` or stay empty? And in which block does the instance acquire its own `items` entry in `__dict__`?

The actual output:

```
['apple'] ['apple'] False ['mango'] [] [] True
```

Why it does that:

The two halves of this trap look almost identical and behave oppositely — that's the whole point of putting them next to each other.

In the first block, `a.items.append("apple")` is *not* an assignment. It's a method call on whatever object `a.items` resolves to. Python does the read (instance miss → class hit, returns the shared list), then calls `.append` on it. The shared list is mutated in place. No write ever happens at the attribute-binding level, so nothing is inserted into `a.__dict__` — the check `"items" in a.__dict__` is False. `b.items` resolves through the same class-level path and finds the same mutated list. This is Trap 37 restated.

In the second block, `c.items = c.items + ["mango"]` is an explicit assignment, and the right-hand side `c.items + ["mango"]` evaluates to a *brand new list*. `list + list` always returns a new list — it never mutates either operand. So the class-level list stays untouched. Then the assignment fires: `c.items = <new list>` inserts `"items"` into `c.__dict__` (Trap 38 mechanism), shadowing the class attribute on `c` only. `d.items` still resolves through the class fallback and finds the unchanged empty list. `Bucket.items` is also still empty.

The pedagogical version of this you'll often see uses `+=`, but `+=` on a list is a special case — it calls `__iadd__` which mutates in place *and* rebinds, so it ends up looking like the `.append` path: shared list mutated, no instance shadow visible from the outside. With an immutable type like `int`, `+=` cleanly behaves like the rebind branch above (because ints have no `__iadd__`, so `a.count += 1` desugars to `a.count = a.count + 1`, which inserts into the instance dict). The trap to remember: any operation that *only* mutates the existing object affects everyone; any operation that *assigns* through the instance creates a shadow visible only to that instance.

What saying this tells the interviewer: "`.append` mutates the shared class-level list and leaks across instances. Assignment — `self.items = something` — always creates an instance shadow and leaves the class object alone. `+=` is the dangerous one: for lists it mutates in place because `__iadd__` exists; for ints it rebinds because there's no in-place op. Test on the actual type before assuming."

Category 10 — Dunder Ordering (4 traps)

Python Interview Prep — 2026 Edition · Learn & Excel · File 05 Guess the Output

Four traps on the order Python uses to look up dunder methods. These are the questions panels reach for when they want to separate "writes classes" from "knows the data model" — `__getattr__` vs `__getattribute__`, the `__eq__` / `__hash__` contract, why `len()` ignores instance attributes, and the subclass-priority rule that flips reflected-operator order. Snippets compile on Python 3.12 / 3.13.

Trap 42 · `__getattr__` vs `__getattribute__` — the recursion trap

Snippet

```
class Order:
    def __init__(self):
        self.amount = 500

    def __getattribute__(self, name):
        print(f"access: {name}")
        return self.amount          # looks innocent

o = Order()
print(o.amount)
```


Your prediction: _____

The actual output:

```
access: amount access: amount access: amount ... (thousands of lines)
RecursionError: maximum recursion depth exceeded
```

Why it does that:

`__getattribute__` runs on **every** attribute access on an instance — it is the primary lookup hook. `__getattr__`, by contrast, only fires when normal lookup has failed (the attribute isn't on the instance, the class, or any base). Two completely different methods, one-letter difference in name.

The body `return self.amount` is itself an attribute access. That access calls `__getattribute__` again. Which evaluates `self.amount` again. Which calls `__getattribute__` again. There is no base case, so the interpreter unrolls until it hits the recursion limit.

The fix is to bypass your own override by calling the unmodified version on `object`:

```
def __getattribute__(self, name):
    print(f"access: {name}")
    return object.__getattribute__(self, name)
```

`object.__getattribute__` is the real lookup machinery — it walks the instance `__dict__`, then the class, then the MRO, and only as a last resort does it dispatch to `__getattr__`. Once you call it, your override is no longer involved, so the recursion stops.

The cousin trap is the opposite confusion — overriding `__getattr__` and expecting it to intercept all accesses. It doesn't. `o.amount` never reaches `__getattr__` because `amount` is on the instance, so normal lookup finds it. `__getattr__` is a fallback, not an interceptor.

What saying this tells the interviewer: "`__getattribute__` is the interceptor, `__getattr__` is the fallback — and any time you override `__getattribute__`, the body has to talk to `object.__getattribute__` or you've written an infinite loop."

Trap 43 · Defining `__eq__` silently makes the class unhashable

Snippet

```
class Money:
    def __init__(self, amount):
        self.amount = amount

    def __eq__(self, other):
        return isinstance(other, Money) and self.amount == other.amount

print({Money(100), Money(100)})
```

Your prediction: _____

The actual output:

```
Traceback (most recent call last): File "<stdin>", line 1, in <module>
TypeError: unhashable type: 'Money'
```

`Money.__hash__` is now `None`, and any attempt to put a `Money` in a `set` or use it as a `dict` key blows up the same way.

Why it does that:

Python guarantees an invariant: if `a == b` then `hash(a) == hash(b)`. If you override `__eq__` without also overriding `__hash__`, the inherited `object.__hash__` (which hashes by identity) would break that invariant — two equal `Money` objects would hash differently. To stop you from quietly corrupting every dict and set you put them in, Python sets `__hash__ = None` on the class for you. `None` means "explicitly not hashable," distinct from "no hash defined."

You see this in the class dict directly:

```
>>> Money.__hash__ is None
True
```

The fix is to define `__hash__` alongside `__eq__`, hashing whatever your equality compares:

```
def __hash__(self):
    return hash(self.amount)
```

Two follow-on facts worth knowing. First, this only happens when **you** define `__eq__` in the class body. Inheriting `__eq__` from a parent doesn't trigger it — Python only kills `__hash__` when it sees the new `__eq__` defined right there. Second, `@dataclass` handles this for you:

`@dataclass(eq=True)` (the default) sets `__hash__ = None`; `@dataclass(frozen=True)` generates both.

What saying this tells the interviewer: "Defining `__eq__` without `__hash__` makes the class unhashable — Python sets `__hash__` to `None` to protect the `a == b` implies `hash(a) == hash(b)` contract."

Trap 44 · `len(obj)` looks up `__len__` on the class, not the instance

Snippet

```
class Bag:
    def __len__(self):
        return 3

b = Bag()
b.__len__ = lambda: 999

print(b.__len__())    # direct call
print(len(b))         # via built-in
```

Your prediction: _____

The actual output:

999 3

Why it does that:

Implicit dunder invocations — `len(x)`, `x + y`, `iter(x)`, `with x:`, `x[k]`, `repr(x)`, almost all of them — skip the instance and look the method up directly on the **type**. The interpreter calls `type(b).__len__(b)`, not `b.__len__()`. Same object semantically; very different lookup paths.

`b.__len__ = lambda: 999` only shoves a function into `b.__dict__`. A direct call `b.__len__()` finds it the normal way (instance dict before class dict, per `__getattr__`'s rules), so it returns 999. But `len(b)` never asks the instance — it asks the class. The class still has `Bag.__len__` from the class body, so it returns 3.

This is by design and predates Python 3. Letting every operator check the instance dict on every call would be too slow, and it would mean monkey-patching one object could change the type system out from under everything else holding a reference to it. Operators are an interface on the **type**, not on the object.

Consequences worth keeping in mind:

- `__init_subclass__`, `__set_name__`, descriptors, metaclasses all live on the class for the same reason — instances can't define them.

- Monkey-patching an operator means patching the class: `Bag.__len__ = lambda self: 999`. That mutation is visible to every existing instance, because the lookup goes through the type.
- A class that wants per-instance operator behaviour has to make the class-level dunder delegate: `def __len__(self): return self._custom_len()`, where `_custom_len` can vary per instance.

What saying this tells the interviewer: "Implicit dunder calls bypass the instance — `len(b)` is `type(b).__len__(b)`. That's why monkey-patching `__len__` onto an instance never affects `len()`."

Trap 45 · Reflected operators — subclass on the right wins, even when the left has `__add__`

Snippet

```
class Amount:
    def __add__(self, other):
        return "Amount.__add__"

class Bonus(Amount):
    def __radd__(self, other):
        return "Bonus.__radd__"

print(Amount() + Bonus())
```

Your prediction: _____

The actual output:

`Bonus.__radd__`

Why it does that:

The normal rule for `a + b` is "try `a.__add__(b)` first; if it returns `NotImplemented`, fall back to `b.__radd__(a)`." That rule has an exception with sharp teeth: if the right operand's type is a **subclass** of the left operand's type, Python tries the right operand's reflected method **first**, before the left's regular method. Only if that returns `NotImplemented` does it fall back to the left.

The reason is overrideability. Without the subclass-priority rule, a subclass that wanted to customise `parent + child` would have no hook — the parent's `__add__` would run before the subclass got a chance. Reflected-first lets the more-specific type win.

Concretely, the dispatch for `Amount() + Bonus()` is:

1. `type(Bonus())` is a subclass of `type(Amount())` and overrides `__radd__` (the parent does not).
2. So Python tries `Bonus.__radd__(b, a)` first — returns `"Bonus.__radd__"`.
3. Because that didn't return `NotImplemented`, the normal `Amount.__add__` never runs.

A few details that trip people up:

- The right operand's class must **override** `__radd__` — inheriting the parent's `__radd__` unchanged does not trigger the swap. Python checks "is `type(b).__radd__` different from `type(a).__radd__`?" before promoting it.
- The rule is symmetric across all binary operators: `__sub__` / `__rsub__`, `__mul__` / `__rmul__`, `__or__` / `__ror__`, and so on.
- For unrelated types — `int + Rupee` where `Rupee` is not a subclass of `int` — the standard order holds: `int.__add__(Rupee)` runs first, returns `NotImplemented`, then `Rupee.__radd__(int)` gets the call. The subclass-priority swap doesn't apply.

The cleanest mental model: subclasses get a veto on operations involving their parent. If they care, they speak first; if they pass (`NotImplemented`), normal order resumes.

What saying this tells the interviewer: "`a + b` is left-first only when the types are unrelated. If the right operand is a subclass of the left and overrides `__radd__`, the subclass wins first — Python designed it that way so subclasses can override operators against their parent type."

Category 11 — Generator pitfalls (5 traps)

Python Interview Prep — 2026 Edition · Learn & Excel · File 05 Guess the Output

Generators are the place where Python's "everything is an object" promise gets weird. The function body looks like ordinary code, but it actually returns a generator object whose execution is paused at every `yield` and resumed by the iterator protocol. That suspension model creates five interview-favourite traps: a `return` that doesn't return, a `StopIteration` that gets upgraded to a `RuntimeError`, a `yield from` that surfaces a sub-generator's return value as its own expression value, a `close()` that injects an exception into the generator body, and a generator expression that captures the source iterable by reference rather than by snapshot. None of these are bugs in CPython — they are language guarantees, mostly nailed down by PEP 380 (Python 3.3) and PEP 479 (default in Python 3.7). Each trap below: snippet, prediction prompt, the actual output, the mechanism, and the stand-out line that signals you have read the data model rather than just shipped `yield` once or twice.

Trap 46 · `return value` inside a generator does not return the value**Snippet**

```
def first_paid(orders):
    for o in orders:
        if o["status"] == "paid":
            return o["id"]
        yield o["id"]

orders = [
    {"id": 1, "status": "pending"},
    {"id": 2, "status": "paid"},
    {"id": 3, "status": "shipped"},
]

g = first_paid(orders)
print(list(g))
```

Your prediction: What does `list(g)` print, and what happened to the `paid` order's id?

The actual output:

```
[1]
```

Why it does that:

A function that contains at least one `yield` anywhere in its body is compiled as a generator function. Calling it returns a generator object; the body does not execute until you iterate. When the body eventually hits a `return` statement (with or without a value), CPython raises `StopIteration` to signal the iterator is exhausted, and the return value is stashed as the `value` attribute on that `StopIteration` instance — it is not yielded, it is not appended, it is not part of `list(...)`. PEP 380 spells this out: "in a generator, `return value` is equivalent to `raise StopIteration(value)`." The driver of the iteration (here, `list`) sees `StopIteration` and stops collecting. The `id` 2 was the return value, but it disappeared into the `StopIteration.value` slot and `list` never reads that slot — it only reads what `__next__` yields.

To actually capture the return value, you have to drive the generator by hand and catch the `StopIteration`:

```

g = first_paid(orders)
collected = []
try:
    while True:
        collected.append(next(g))
except StopIteration as e:
    final = e.value          # 2
print(collected, final)    # [1] 2

```

Or use `yield from`, which is the one syntax that does surface the return value of a sub-generator as an expression (see Trap 48).

What saying this tells the interviewer: "`return` in a generator is sugar for `raise StopIteration(value)` — the value lives on the exception, not on the iteration, so `list()` cannot see it. You read it via `try/except StopIteration as e: e.value`, or via `yield from`." That single sentence proves you have read PEP 380 rather than learnt generators from a 5-minute YouTube clip.

Trap 47 · `StopIteration` raised inside a generator body becomes `RuntimeError` (PEP 479)

Snippet

```

def take_until_empty(it):
    while True:
        yield next(it)

source = iter(["razorpay", "swiggy", "zerodha"])
for name in take_until_empty(source):
    print(name)

```

Your prediction: Three names print and the loop ends cleanly, right?

The actual output:

```

razorpay swiggy zerodha Traceback (most recent call last): ...
RuntimeError: generator raised StopIteration

```

Why it does that:

PEP 479 — adopted as a `__future__` import in Python 3.5, opt-out in 3.6, and the only behaviour from Python 3.7 onward — changed how `StopIteration` is handled when it bubbles up out of a

generator frame. Before PEP 479, a `StopIteration` raised inside the body (here, by `next(it)` when `it` was exhausted) would propagate through the generator and be caught silently by whatever loop was driving it, ending the generator as if it had simply run out. That was a quiet, hard-to-debug failure mode: a generator could be terminated by an unrelated `StopIteration` from anywhere inside its body, including from deeply nested helpers. After PEP 479, the interpreter watches for this exact case and converts that escaping `StopIteration` into a `RuntimeError("generator raised StopIteration")`. The original `StopIteration` is chained as the `__context__` (it was the exception in flight when the new `RuntimeError` was raised), which preserves the trace for debugging.

Three names print because the first three `next(it)` calls return values which the generator yields, and the `for` loop consumes them. The fourth `next(it)` raises `StopIteration` (the source iterator is empty), that exception escapes the generator's frame, and the interpreter rewrites it into `RuntimeError`. The fix is to catch the `StopIteration` at the call site and `return` instead:

```
def take_until_empty(it):
    while True:
        try:
            yield next(it)
        except StopIteration:
            return          # ends the generator cleanly
```

Or, better, iterate `it` directly with `for`, which handles the termination for you.

What saying this tells the interviewer: "PEP 479 made `StopIteration` leaking out of a generator into a `RuntimeError` — default since 3.7. It plugs the silent-termination bug where a stray `StopIteration` from a helper would end the generator without anyone noticing. The fix is to catch and `return`, or to use a `for` loop instead of bare `next`." Citing PEP 479 by number and explaining the silent-failure motivation marks you as someone who has shipped generators in production, not just written them in a notebook.

Trap 48 · `yield from` surfaces the sub-generator's `return` value as its own expression value

Snippet


```
def inner():
    yield 1
    yield 2
    return "inner done"

def outer():
    value = yield from inner()
    yield value

print(list(outer()))
```

Your prediction: What sits in that final list?

The actual output:

```
[1, 2, 'inner done']
```

Why it does that:

`yield from` is the one place in the language where a generator's return value escapes the `StopIteration.value` jail and becomes a normal expression result. PEP 380 defined the semantics: `yield from <iterable>` delegates iteration to the sub-iterable, forwarding every value the sub-iterable yields to the outer caller. When the sub-iterable is a generator and it executes a `return value`, the `StopIteration(value)` it raises is caught by the `yield from` machinery, and the `value` attribute becomes the value of the `yield from` expression itself.

So `value = yield from inner()` does two things over time: while `inner()` is yielding, those values pass straight through to the consumer of `outer()` — that's where `1` and `2` come from in the list. When `inner()` runs `return "inner done"`, that does not bubble up as `StopIteration`; instead, the `yield from` resolves to `"inner done"` and assigns it to `value`. The next line `yield value` then emits `"inner done"` as the third item in the list.

This is what makes `yield from` useful for coroutines, recursive tree walks, and any case where a sub-generator needs to return a final accumulated result to its caller while also yielding intermediate values. It is also the only syntax-level way to read a generator's return value without writing the `try/except StopIteration as e: e.value` boilerplate from Trap 46.

What saying this tells the interviewer: *"`yield from` does two jobs — it forwards yields from the sub-generator, and it captures the sub-generator's `return` value as the expression's value. That's PEP 380, and it's why coroutine code reads cleanly without manual `StopIteration` handling."* Pointing at PEP 380 by name lands; mentioning that `yield from` also forwards `send`, `throw`, and `close` to the sub-generator is the bonus line that pushes the rating from "knows generators" to "knows the iterator protocol cold."

Trap 49 · Yielding inside the `except GeneratorExit` block raises `RuntimeError`

Snippet

```
def listener():
    try:
        while True:
            data = yield
            print("got", data)
    except GeneratorExit:
        yield "goodbye"      # cleanup attempt

g = listener()
next(g)                    # prime the generator
g.send("ping")
g.close()
```

Your prediction: Does `g.close()` walk away quietly, or does it raise?

The actual output:

```
got ping
Traceback (most recent call last): ... RuntimeError: generator
ignored GeneratorExit
```

Why it does that:

A generator's `close()` method works by injecting a `GeneratorExit` exception into the suspended generator at its current `yield` point. The expected contract is that the generator either lets the exception propagate (which terminates the generator) or catches it, does cleanup, and then either `return`s or re-raises. What it absolutely must not do is yield another value, because the whole point of `close()` is to terminate iteration — yielding would mean the iteration is not over, which contradicts the close request. The interpreter enforces this: if the generator catches `GeneratorExit` and then executes a `yield` (or `yield from`), CPython raises `RuntimeError("generator ignored GeneratorExit")` at the closing call site.

The same rule applies if you swallow `GeneratorExit` without re-raising and the generator function simply falls through and returns — that is fine, because returning is the normal way to exit. The constraint is specifically against re-suspending the generator. The cleanup pattern that does work is:

```
def listener():
    try:
        while True:
            data = yield
            print("got", data)
    except GeneratorExit:
        print("cleaning up")    # any side-effect is fine
        # no yield here; just fall through or `return`
```

This is also why `contextlib.contextmanager`-style generators must have their `yield` outside the `finally` — yielding after a `GeneratorExit` was thrown in is the exact failure mode that the warning is designed to catch.

What saying this tells the interviewer: "`close()` throws `GeneratorExit` at the yield point. You can catch it for cleanup, but you cannot yield again — the interpreter promotes that to `RuntimeError: generator ignored GeneratorExit`. That's the rule that keeps `close()` from being silently meaningless." Pairing this with "and that's why `contextmanager` requires the `yield` to live outside any `finally` block" wins the round, because it links the language rule to the standard-library design that respects it.

Trap 50 · Generator expressions capture the source iterable by reference, not by snapshot

Snippet

```
orders = [100, 200, 300]
g = (x * 2 for x in orders)
orders.append(400)
orders[0] = 999
print(list(g))
```

Your prediction: Does the generator see the mutation, or does it iterate over the snapshot that existed when the expression was written?

The actual output:

```
[1998, 400, 600, 800]
```

Why it does that:

This trap separates "I read PEP 289" from "I learnt that gen exps and list comps are the same with different brackets." A list comprehension `[x * 2 for x in orders]` materialises the entire list

immediately — it iterates `orders` to completion at the moment the expression runs, builds a new list, and is then fully detached from `orders`. A generator expression `(x * 2 for x in orders)` does something quite different. PEP 289 specifies that the outermost iterable is evaluated immediately and stored on the generator object — but only the *reference* is stored, not a copy. The actual iteration happens lazily: each `next()` call advances the generator one step, which means it does `iter(orders)` once at the start, and then walks that iterator at consumption time.

In our snippet, the line `g = (x * 2 for x in orders)` calls `iter(orders)` and stores the resulting list-iterator. At this moment, the list object is `[100, 200, 300]`. Then `orders.append(400)` mutates the same list in place — it does not create a new list — so the list object the iterator points at now has four elements. Then `orders[0] = 999` mutates index 0 to `999`. When `list(g)` consumes the generator, the iterator walks the (now-mutated) list and yields `999 * 2, 200 * 2, 300 * 2, 400 * 2`.

The general rule: if you build a generator expression over a mutable container and someone mutates that container before the generator is exhausted, the generator sees the mutated state. The fix, if you want snapshot semantics, is to materialise immediately: `g = iter([x * 2 for x in orders])`, or to copy the source upfront: `g = (x * 2 for x in list(orders))`. The same trap applies to file handles, dict views, and any iterable whose state is mutable between generator creation and generator consumption.

What saying this tells the interviewer: "A generator expression calls `iter()` on its outermost iterable eagerly, but it walks that iterator lazily — so mutations to the underlying container between creation and consumption are visible. List comprehensions materialise everything up front and so they don't see the mutation. If I need snapshot semantics from a gen exp, I wrap the source in `list(...)` first." This answer telegraphs three things at once — you know PEP 289's "outermost iterable is evaluated immediately" rule, you know the difference between eager iterable evaluation and lazy iteration, and you know the fix without reaching for a library.
